

StartupSage — Phase 1 Kickoff

Week 1 Day-by-Day Task List

You + Ganesh + Dhaneshwari + Srushti

BEFORE ANYTHING ELSE — YOU DO THIS FIRST (Day 1 Morning, Before Calling Anyone)

These are your setup tasks. Interns cannot start until these exist. Do these in order.
Takes roughly 2–3 hours.

☐ Step 1: Create GitHub Organisation

Go to github.com → New Organisation → Name: "buildpunetogether" (or similar)

Create repo inside it: "startupsage"

Visibility: Private

Add a basic README.md with one line: "StartupSage — India's First Experiential Entrepreneurship Platform"

☐ Step 2: Create branch structure

Main branch: main (protected — no one pushes here directly including you)

Create these branches from main:

dev ← all interns merge into this, you review before main

ganesh/design ← Ganesh's Figma links + design exports

dhanesh/backend ← Dhaneshwari's Flask code

srushti/ai ← Srushti's Python modules

☐ Step 3: Create Supabase project

Go to supabase.com → New Project

Name: startupsage

Region: South Asia (Mumbai) — closest to your users

Save: Project URL + Anon Key + Service Role Key (you'll need these)

☐ Step 4: Create Notion workspace

New page: "StartupSage Intern Board"

Create a simple table with columns:

Task | Assigned To | Status (To Do / In Progress / Done) | Due Date | Submission Link

Pre-fill Week 1 tasks for each intern (copy from this document)

☐ Step 5: Invite everyone

GitHub repo: invite ganesh, dhaneshwari, srushti as Collaborators

Notion board: share link with all three

Create WhatsApp group: "StartupSage Team 🚀"

Members: You, Ganesh, Dhaneshwari, Srushti

Pin this message as the first message:

Welcome to StartupSage team 🚀

Daily update format (post by 7 PM every working day):

✅ Done: [what you completed today]

🔄 Working on: [what's in progress]

❌ Blocked: [anything stopping you — tag me]

Rules:

→ Stuck for more than 2 hours? Tag me immediately. Don't wait.

→ No update by 7 PM = I'll call you.

→ All code goes on your branch. Never push to main or dev.

→ Task done = update Notion card to Done + paste submission link.

□ Step 6: Create base folder structure in the repo

Push this initial structure to main:

startupsage/

```
└─ backend/
|   └─ app/
|       └─ __init__.py
|       └─ routes/
|       └─ services/
|       └─ models/
|       └─ utils/
|   └─ tests/
|   └─ requirements.txt    ← empty for now
|   └─ .env.example       ← with placeholder keys
|   └─ run.py             ← Flask entry point stub
└─ ai/
    └─ scenario_engine/
    └─ scoring/
    └─ prompts/
    └─ tests/
└─ mobile/                ← Flutter project goes here (Ganesh creates)
└─ designs/               ← Ganesh pastes Figma links here in README.md
└─ README.md
```

Push this and confirm everyone can see it on GitHub.

☐ Step 7: Call each intern individually (30 min each, same day)

Order: Dhaneshwari → Srushti → Ganesh

(Backend and AI first — they have the most setup to do)

Script for each call is below in their respective Day 1 sections.

HOW SUBMISSION WORKS (Tell Every Intern This on Day 1 Call)

Every task has a specific deliverable. Vague "I did it" is not acceptable. Each deliverable must be submitted as one of these:

For Ganesh (Design):

- Figma link shared with you (set to "Anyone with link can view")
- Screenshot pasted in WhatsApp group
- Notion card updated to Done + Figma link pasted in submission column

For Dhaneshwari (Backend):

- Code pushed to her branch (dhanesh/backend)
- Postman screenshot showing the endpoint working (request + response visible)
- Notion card updated to Done + GitHub commit link pasted

For Srushti (AI):

- Python file pushed to her branch (srushti/ai)
- Test output screenshot (terminal showing tests passing)
- Notion card updated to Done + GitHub commit link pasted

For you:

- Push to main or dev branch
 - Update Notion card
-
-

DAY 1 — MONDAY

Theme: Setup. Everyone gets their environment running.

Goal: By end of Day 1, everyone has their tools installed and can access the repo.

YOU — Day 1

Morning (2–3 hrs): Complete all 7 setup steps above before any call.

Then: Kickoff calls (30 min each)

12:00 PM → Dhaneshwari

12:45 PM → Srushti

1:30 PM → Ganesh

Call script (same for all three):

1. "Here's what we're building" — show them the StartupSage flow diagram
(share screen, walk through Stage 0 → Stage 8 in 5 minutes)
2. "Here's your role" — say it clearly:

Ganesh: "You own everything the student sees. Figma first, Flutter second."

Dhaneshwari: "You own the Flask backend. Every API endpoint is yours."

Srushti: "You own the AI brain. Scenario logic, scoring, Claude API prompts."
3. "Here's your Day 1 task" — walk through exactly what's below for them
4. "Here's how you submit" — show them the Notion board and WhatsApp format
5. "Here's the rule" — stuck 2 hours? Message me. Don't sit blocked.
6. Ask: "Any setup issues right now?" Fix them on the call if possible.

Afternoon: While interns set up, you start the Supabase schema.

Create these tables (copy schema from the intern plan document):

schools

students

simulation_sessions
stage_decisions
offline_challenge_submissions
skills_scores
founder_badges
weekly_missions
founder_clips
admin_users

Don't create all columns perfectly — get the core ones right.

You'll refine as Dhaneshwari starts hitting the DB.

End of Day 1: Post your own update in WhatsApp (lead by example):

✅ Done: GitHub repo created, Supabase schema started, all 3 kickoff calls done

🔄 Working on: Finishing schema, .env.example file

❌ Blocked: Nothing

GANESH — Day 1

KICKOFF CALL WITH YOU: 1:30 PM

(You explain the product and his role — 30 min)

After the call — Day 1 tasks:

Task: Environment Setup

Step 1: Accept GitHub invite → clone the repo

```
git clone https://github.com/[org]/startupsage.git
```

```
cd startupsage
```

```
git checkout -b ganesh/design
```

git push origin ganesh/design

Step 2: Install Figma (desktop app — not browser, desktop is faster)

Download from figma.com/downloads

Create a free account if you don't have one

Create a new Figma file called "StartupSage — Design System"

Step 3: Install Flutter

Download from flutter.dev → Windows installer

Run: flutter doctor

Fix any issues flutter doctor shows (Android Studio, SDK, etc.)

Run: flutter --version (should show 3.x.x)

If flutter doctor shows errors — screenshot and send in WhatsApp

Step 4: Read these two documents fully before tomorrow

→ The StartupSage 3-phase plan (link in Notion)

→ Your role section in the Intern Plan document (link in Notion)

Takes about 45 minutes. No skimming.

SUBMISSION FOR DAY 1:

Screenshot of: flutter --version in terminal (shows Flutter installed)

Screenshot of: blank Figma file created with your name

Paste both in WhatsApp group with your 7 PM update

Update Notion card: "Day 1 Setup" → Done

DHANESHWARI — Day 1

KICKOFF CALL WITH YOU: 12:00 PM

(You explain the product and her role — 30 min)

After the call — Day 1 tasks:

Task: Environment Setup

Step 1: Accept GitHub invite → clone the repo

```
git clone https://github.com/[org]/startupsage.git
```

```
cd startupsage
```

```
git checkout -b dhanesh/backend
```

```
git push origin dhanesh/backend
```

Step 2: Set up Python environment

Make sure Python 3.11+ is installed: `python --version`

Navigate to backend/ folder

Create virtual environment:

```
python -m venv venv
```

Windows: `venv\Scripts\activate`

Mac/Linux: `source venv/bin/activate`

Step 3: Install Flask and core packages

```
pip install flask flask-cors supabase python-dotenv resend pytest
```

```
pip freeze > requirements.txt
```

Push requirements.txt to her branch:

```
git add requirements.txt
```

```
git commit -m "add: initial requirements"
```

```
git push origin dhanesh/backend
```

Step 4: Copy .env.example to .env (already in repo from your setup)

Fill in the Supabase URL and keys (you share these in the call)

NEVER push the .env file — it's in .gitignore

Step 5: Create the Flask app skeleton

In backend/run.py write:

```
from app import create_app  
  
app = create_app()  
  
if __name__ == '__main__':  
    app.run(debug=True, port=5000)
```

In backend/app/__init__.py write:

```
from flask import Flask  
  
from flask_cors import CORS  
  
def create_app():  
    app = Flask(__name__)  
  
    CORS(app)  
  
    @app.route('/health')  
    def health():  
        return {'status': 'ok', 'service': 'startupsage-api'}  
  
    return app
```

Run it: python run.py

Open browser: <http://localhost:5000/health>

Should see: {"service": "startupsage-api", "status": "ok"}

Step 6: Read these documents fully today

- The StartupSage 3-phase plan
- Her role section in the Intern Plan
- API endpoints list in the Intern Plan (her sections)

SUBMISSION FOR DAY 1:

Screenshot of: <http://localhost:5000/health> response in browser

Screenshot of: requirements.txt file contents

Screenshot of: her branch showing on GitHub

Paste all 3 in WhatsApp with 7 PM update

Update Notion card: "Day 1 Setup" → Done

Push all code to dhanesh/backend branch

SRUSHTI — Day 1

KICKOFF CALL WITH YOU: 12:45 PM

(You explain the product and her role — 30 min)

After the call — Day 1 tasks:

Task: Environment Setup + Deep Read

Step 1: Accept GitHub invite → clone the repo

```
git clone https://github.com/[org]/startupsage.git
```

```
cd startupsage
```

```
git checkout -b srushti/ai
```

```
git push origin srushti/ai
```

Step 2: Set up Python environment for AI folder

Navigate to ai/ folder

```
python -m venv venv
```

Activate it (same as above)

```
pip install anthropic scikit-learn numpy pytest python-dotenv
```

pip freeze > requirements.txt

Push to her branch

Step 3: Copy .env and add API keys

ANTHROPIC_API_KEY — you share this in the call

Test it works:

Create ai/tests/test_connection.py:

```
import anthropic, os

from dotenv import load_dotenv

load_dotenv()

client = anthropic.Anthropic(api_key=os.getenv('ANTHROPIC_API_KEY'))

message = client.messages.create(

    model='claude-sonnet-4-6',

    max_tokens=50,

    messages=[{'role': 'user', 'content': 'Say hello in 5 words'}]

)

print(message.content[0].text)
```

Run it. Should print a 5-word hello.

Step 4: Read everything — this is the most important Day 1 task

- The full StartupSage 3-phase plan (all stages, all features)
- Her role section in the Intern Plan
- The 8 Wall scenarios and all stage decision flows

After reading: write a 1-page summary in a Google Doc:

"Here is how I understand the simulation logic end to end"

"Here is what my modules need to do and in what order"

"Here are questions I have before I start coding"

Share the Google Doc link in WhatsApp today.

This is not busywork. It catches misunderstandings before code is written.

SUBMISSION FOR DAY 1:

Screenshot of: test_connection.py running and printing a response

Screenshot of: her branch on GitHub

Google Doc link: 1-page simulation understanding summary

Paste all in WhatsApp with 7 PM update

Update Notion card: "Day 1 Setup" → Done

DAY 2 — TUESDAY

Theme: First real work. Everyone writes their first meaningful output.

YOU — Day 2

Morning: Review Day 1 submissions from all 3 interns

Check each submission in WhatsApp

If someone didn't submit → call them first thing, find out why

Fix any setup blockers (this is your top priority today)

Finish Supabase schema:

Complete all column definitions for all 10 tables

Add Row Level Security policies:

students: can only read their own row

simulation_sessions: can only read sessions where studentId = auth.uid

schools: readable by admin only

Share schema screenshot in Notion for Dhaneshwari's reference

Write .env.example (push to repo main):

SUPABASE_URL=

SUPABASE_SERVICE_ROLE_KEY=

SUPABASE_ANON_KEY=

ANTHROPIC_API_KEY=

RESEND_API_KEY=

FLASK_SECRET_KEY=

FLASK_ENV=development

Write CONTRIBUTING.md and push to repo main:

How to create a branch

How to commit (format: "add: feature name" / "fix: bug description" / "update: what changed")

How to raise a PR

PR checklist: does it work? is it tested? is it documented?

GANESH — Day 2

Task: Design System in Figma (full day)

This is the foundation. Every screen you design later uses these components.

Build it once. Use it everywhere. Don't rush this.

Step 1: Colour Palette (30 min)

Create a "Colours" frame in Figma

Define these as Figma Styles (right panel → Style → + button):

Primary:

BPT Orange: #F97316 ← main CTA colour

Orange Light: #FED7AA ← selected states, highlights

Orange Dark: #C2410C ← pressed/hover states

Neutral:

Background: #FAF9F9 ← app background (warm white, not harsh)

Card White: #FFFFFF ← card surfaces

Border: #E7E5E4 ← card borders, dividers

Text Primary: #1C1917 ← headlines, important text

Text Secondary: #78716C ← supporting text

Text Muted: #A8A29E ← placeholder text, disabled states

Semantic:

Success: #16A34A

Warning: #D97706

Error: #DC2626

Info: #3B82F6

Domain colours (for domain badges):

Healthcare: #10B981 (emerald)

Education: #3B82F6 (blue)

Finance: #8B5CF6 (violet)

Agriculture: #84CC16 (lime)

Environment: #06B6D4 (cyan)

Gaming: #EC4899 (pink)

AI: #A855F7 (purple)

Step 2: Typography (30 min)

Define Text Styles in Figma:

Heading/H1: Plus Jakarta Sans, 28px, Bold, #1C1917

Heading/H2: Plus Jakarta Sans, 22px, SemiBold, #1C1917

Heading/H3: Plus Jakarta Sans, 18px, SemiBold, #1C1917

Body/Large: Inter, 16px, Regular, #1C1917

Body/Default: Inter, 14px, Regular, #1C1917

Body/Small: Inter, 12px, Regular, #78716C

Caption: Inter, 11px, Regular, #A8A29E

CTA/Button: Inter, 14px, SemiBold

Step 3: Core Components (3 hrs)

Build each as a Figma Component (Ctrl+Alt+K):

Button — Primary:

Background: BPT Orange, border-radius: 12px, padding: 14px 24px

Text: white, CTA/Button style

States: Default / Pressed (Orange Dark bg) / Disabled (50% opacity)

Button — Secondary:

Background: white, border: 1.5px Orange, border-radius: 12px

Text: Orange, CTA/Button style

States: Default / Pressed / Disabled

Domain Badge:

Pill shape (border-radius: 999px), padding: 4px 10px

Background + text colour based on domain (use domain colours above)

Font: Body/Small, SemiBold

Make one base component, use variants for each domain

Stage Badge:

Same as Domain Badge but colours:

IDEA: slate-100 bg, slate-600 text

PRE_MVP: yellow-100 bg, yellow-700 text

MVP: blue-100 bg, blue-700 text

TRACTION: green-100 bg, green-700 text

SCALING: orange-100 bg, orange-700 text

Card:

Background: white, border-radius: 16px

Border: 1px solid #E7E5E4

Shadow: 0 1px 3px rgba(0,0,0,0.08)

Padding: 16px

Use this as a frame for all content cards

Bottom Navigation Bar:

5 tabs: Home | Stages | Journal | Glossary | Pulse

Height: 80px (includes safe area padding at bottom)

Active tab: BPT Orange icon + label

Inactive: Text Muted icon + label

Background: white, top border: 1px Border colour

Input Field:

Height: 48px, border-radius: 10px

Border: 1.5px #E7E5E4, focus border: BPT Orange

Background: white

Placeholder text: Text Muted style

Progress Bar (Stage Progress):

9 dots in a row (one per stage)

Completed dot: BPT Orange filled circle, 10px

Current dot: Orange outline + pulsing ring animation note

Upcoming dot: Border colour filled, 8px

Step 4: Export

Export colour palette as PNG (screenshot)

Share Figma file link in WhatsApp group

SUBMISSION FOR DAY 2:

Figma link with: colours, typography, and all 8 components built

Screenshot of the Figma components panel showing all components

WhatsApp update by 7 PM

Notion card: "Day 2 — Design System" → Done + Figma link

DHANESHWARI — Day 2

Task: Supabase Connection + Student Registration API

Step 1: Connect Flask to Supabase (1 hr)

Create backend/app/utils/supabase_client.py:

```
import os
```

```
from supabase import create_client, Client
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
url = os.getenv('SUPABASE_URL')
```

```
key = os.getenv('SUPABASE_SERVICE_ROLE_KEY')
```

if not url or not key:

raise ValueError('SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY must be set in .env')

supabase: Client = create_client(url, key)

Test: In Python shell, import supabase_client and run:

```
from app.utils.supabase_client import supabase
result = supabase.table('schools').select('*').limit(1).execute()
print(result)
```

Should return: data=[] count=None (empty table, no error)

Step 2: Create school registration endpoint (2 hrs)

Create backend/app/routes/admin_routes.py:

POST /api/admin/schools

Body:

```
{
  "schoolName": "Delhi Public School",
  "city": "Pune",
  "board": "CBSE",
  "principalName": "Mrs. Sharma",
  "principalEmail": "principal@dps.edu.in",
  "packageTier": "starter"
}
```

Logic:

1. Validate all required fields present — return 400 if missing
2. Check school name doesn't already exist — return 409 if duplicate
3. Auto-generate school code:
initials = first letters of each word in school name (max 3 letters)
year = current year (2026)
code = f"BPT-{initials.upper()}-{year}"
Example: "Delhi Public School" → BPT-DPS-2026
4. Insert into schools table
5. Return: { schoolId, schoolCode, schoolName, packageTier, createdAt }

Error handling:

Wrap everything in try/except

Log errors: `print(f"[ERROR] create_school: {str(e)}")`

Return { "error": "descriptive message", "code": "ERROR_CODE" } with correct status

Step 3: Write 3 tests in backend/tests/test_admin.py

Test 1: Create school successfully → verify response has schoolCode

Test 2: Missing required field → verify 400 response

Test 3: Duplicate school name → verify 409 response

Run: `pytest tests/test_admin.py -v`

SUBMISSION FOR DAY 2:

Code pushed to dhanesh/backend branch

Screenshot: POST /api/admin/schools working in Postman

Show: request body on left, response with schoolCode on right

Screenshot: `pytest tests/test_admin.py -v` passing (all 3 tests green)

WhatsApp update by 7 PM

Notion card: "Day 2 — School Registration API" → Done + commit link

SRUSHTI — Day 2

Task: Scenario Assignment Engine (Stage 5 Wall Logic)

You read everything yesterday. Today you write the first real AI module.

This is the most important function in the simulation — it decides which Wall scenario a student faces based on all their Stage 0–4 decisions.

File to create: ai/scenario_engine/scenario_engine.py

Step 1: Define the data structures (30 min)

Write these at the top of the file with full comments:

```
DECISION_INPUT = {  
    'domain': str,          #  
    'healthcare'|'education'|'finance'|'agriculture'|'environment'|'gaming'|'ai'  
    'co_founder_choice': str, # 'technical'|'sales'|'domain_expert'|'solo'  
    'equity_split': float,   # student's % e.g. 60.0 means student keeps 60%  
    'mvp_type': str,        # 'paper'|'landing_page'|'canva'|'product'  
    'budget_allocation': dict, # {'tech': 40, 'marketing': 30, 'ops': 20, 'hiring': 10}  
    'business_model': str    # 'subscription'|'one_time'|'freemium'  
}
```

```
SCENARIO_OUTPUT = {  
    'scenario_id': int,      # 1–8  
    'scenario_title': str,  
    'scenario_description': str, # 2–3 sentences, written for the student  
    'trigger_reason': str    # internal only — why this scenario fired
```

}

Step 2: Define the 8 scenarios as constants (30 min)

SCENARIOS = {

1: {

'title': 'Your CTO Just Quit',

'description': 'Your technical co-founder received a ₹18LPA offer from a MNC. '

'They handed in their notice this morning. You have 30 days of runway left '

'and a product that only they understood.',

'trigger': 'technical co-founder chosen'

},

2: {

'title': 'You Are Burning Out',

'description': 'You have not slept properly in 3 weeks. You are the only person '

'doing everything — building, selling, supporting customers. '

'Your health is declining and you have ₹80,000 left in the bank.',

'trigger': 'solo founder'

},

3: {

'title': 'A Funded Competitor Just Launched',

'description': 'A startup with ₹2 Crore in VC funding launched in your market yesterday. '

'They are offering the same product free for 3 months. '

'Your early users are getting emails from them.',

'trigger': 'B2C model chosen'

},

4: {

'title': 'Your Marketing Channel Died',

'description': 'The one channel that was working — Instagram ads — just stopped converting. '

'Your CAC has doubled in 2 weeks and you have spent 50% of your budget '

'on marketing already. ₹70,000 left. '

'trigger': 'heavy marketing allocation (>40%)'

},

5: {

'title': 'Your Developer Missed Every Deadline',

'description': 'The developer you hired 6 weeks ago has delivered nothing usable. '

'Three client demos have gone badly. One client is threatening a refund. '

'You are 6 weeks behind your planned launch. '

'trigger': 'weak team hire pattern'

},

6: {

'title': 'App Store Rejected Your App',

'description': 'Google Play has rejected your app for the second time — '

'policy violation in how you handle user data. '

'Fixing it will take 3 weeks minimum and cost ₹40,000 in developer time. '

'Your launch date has passed. '

'trigger': 'gaming domain chosen'

},

7: {

'title': 'Monsoon Failed. Your Users Have No Money. '

'description': 'This season has seen the worst rainfall in 10 years in your target region. '

'The farmers you built for have no income this quarter. '

'All 3 of your paying customers have asked to pause their subscriptions. '

'trigger': 'agriculture domain chosen'

},

```

8: {
    'title': 'Your Hospital Partner Backed Out',
    'description': 'The hospital that agreed to pilot your product has pulled out — '
        'their legal team flagged a compliance concern with how you store patient
data. '
        'They need a full HIPAA/DPDP audit before proceeding. '
        'That takes 2 months and ₹1.5 Lakh.',
    'trigger': 'healthcare domain chosen'
}
}

```

Step 3: Write the assignment function (1 hr)

```
def assign_wall_scenario(decisions: dict) -> dict:
```

```
    """
```

```
    Assigns a Wall scenario (Stage 5) based on student decisions from Stages 0-4.
```

```
    Priority order (check in this order, first match wins):
```

1. Domain-specific scenarios (domain locks in scenario for 3 domains)
2. Co-founder scenarios (solo or technical)
3. Budget/model scenarios (heavy marketing or B2C)
4. Default (scenario 5 — weak team hire)

```
    Args:
```

```
        decisions: dict matching DECISION_INPUT structure
```

```
    Returns:
```

```
        dict matching SCENARIO_OUTPUT structure
```

```
    """
```

```
domain = decisions.get('domain', "")
co_founder = decisions.get('co_founder_choice', "")
budget = decisions.get('budget_allocation', {})
model = decisions.get('business_model', "")

marketing_pct = budget.get('marketing', 0)

# Domain-specific triggers (highest priority)
if domain == 'gaming':
    scenario_id = 6
elif domain == 'agriculture':
    scenario_id = 7
elif domain == 'healthcare':
    scenario_id = 8

# Co-founder triggers
elif co_founder == 'technical':
    scenario_id = 1
elif co_founder == 'solo':
    scenario_id = 2

# Budget/model triggers
elif model in ('one_time', 'freemium') and domain not in
('gaming','agriculture','healthcare'):
    scenario_id = 3 # B2C-adjacent model → competitor
elif marketing_pct > 40:
    scenario_id = 4 # heavy marketing → channel failure
```



```

# Default

else:

    scenario_id = 5

scenario = SCENARIOS[scenario_id]

return {

    'scenario_id': scenario_id,

    'scenario_title': scenario['title'],

    'scenario_description': scenario['description'],

    'trigger_reason': scenario['trigger']

}

```

Step 4: Write 10 unit tests (1 hr)

File: ai/tests/test_scenario_engine.py

Test each scenario fires for the right input.

Also test: does it return the right keys? Does it handle missing fields gracefully?

Example test structure:

```

def test_gaming_domain_triggers_scenario_6():

    decisions = {

        'domain': 'gaming',

        'co_founder_choice': 'technical',

        'budget_allocation': {'tech': 40, 'marketing': 20, 'ops': 20, 'hiring': 20},

        'business_model': 'freemium'

    }

    result = assign_wall_scenario(decisions)

    assert result['scenario_id'] == 6

```

```
assert 'App Store' in result['scenario_title']
```

Run: `pytest ai/tests/test_scenario_engine.py -v`

All 10 must pass before submitting.

SUBMISSION FOR DAY 2:

Code pushed to srushti/ai branch

Screenshot: pytest running, all 10 tests green

WhatsApp update by 7 PM with screenshot

Notion card: "Day 2 — Scenario Engine" → Done + commit link

DAY 3 — WEDNESDAY

Theme: First real features. Everyone builds on top of Day 2 work.

YOU — Day 3

Morning: Review all Day 2 submissions

Check Postman screenshot from Dhaneshwari — does the API look right?

Check Figma from Ganesh — does the design system feel on-brand?

Check test output from Srushti — are all 10 tests passing?

Give feedback in WhatsApp (keep it specific: "the card border radius should be 16px not 8px" not "looks good")

Afternoon: Build the internal test harness

This is a minimal HTML file (not Next.js — just plain HTML + fetch)

Save as: `startupsage/test-harness/index.html`

What it does:

A simple form UI where you can:

- Enter a school code + student name → call POST /api/students/register
- Select a domain + enter decisions → call POST /api/sessions/start
- Save stage decisions → call POST /api/sessions/:id/save-stage
- View the session state → call GET /api/sessions/:id/progress
- Trigger scenario engine → call POST /api/scenarios/assign

This is your testing tool. It doesn't need to look good.

It needs to let you call every API endpoint manually without Postman.

Plain HTML, inline CSS, fetch() for API calls. That's it.

Why: Once all interns' APIs are working, you test the full simulation flow from this file without needing Flutter to be ready.

GANESH — Day 3

Task: Design Stage 0 (The Spark) — All States

Now the design system is ready. Start designing actual screens.

Mobile dimensions: 390 × 844px (iPhone 14 Pro — Android equivalents similar)

Frame 1: Splash / Opening Screen

Full black background

Center: StartupSage wordmark (simple bold white text for now — logo later)

Below: "India's First Founder Simulation"

Subtle: small BPT logo at bottom

Note in Figma: "Voice narration plays here: Every company started with a problem..."

Frame 2: Domain Selection Screen (Default — nothing selected)

Top: Stage 0 progress dot active (9 dots, first one orange)

Headline: "Choose your mission" (H2, Text Primary)

Sub: "What problem do you want to solve?" (Body/Default, Text Secondary)

7 domain cards in a 2-column grid (last card centered if odd):

Each card (use your Card component):

Domain emoji (large, 32px) — top center

Domain name (Body/Large, SemiBold)

1-line description (Body/Small, Text Secondary, max 2 lines)

Domain descriptions:



Healthcare — "Fix how India accesses medical care"



Agriculture — "Help farmers grow more and earn more"



Education — "Make quality learning reach every student"



Environment — "Solve climate problems for tomorrow's India"



Finance — "Make money work for people who have less of it"



Gaming — "Build games that entertain millions"



AI — "Use artificial intelligence to solve real problems"

No card selected → [Begin my journey] button: disabled (grey)

Frame 3: Domain Selection Screen (One Selected)

Same as Frame 2 but:

Selected card: Orange Light background, Orange border 2px, checkmark top-right

All other cards: unchanged

[Begin my journey] button: Active (BPT Orange)

Frame 4: Problem Discovery (after domain selected)

This is the first activity inside Stage 0

Shows 3 scenario cards for their chosen domain

Design for Healthcare domain as the example:

Card 1: "Sunita, 34, schoolteacher in Nagpur. Can't afford specialist care for her daughter."

Card 2: "Raj, 52, runs a pharmacy in rural Bihar. Has no way to verify prescription authenticity."

Card 3: "ASHA worker in UP covering 40 villages. Has no digital records, everything on paper."

Each card: name + age + location chip + 2-sentence problem

[This is the problem I want to solve] button below each card

Frame 5: Problem Selected + Emotional Hook

After student selects a problem

Full screen moment — important, make it feel significant:

Large quote marks (decorative)

The selected problem text (Body/Large, centered)

Below: "This is the problem you've chosen to solve."

Below: "It will matter much more at 2 AM when things go wrong." (italic, Text Secondary)

[Yes, let's build →] CTA (full width, BPT Orange)

SUBMISSION FOR DAY 3:

5 frames in Figma: Splash + Domain Default + Domain Selected +

Problem Discovery + Problem Selected

Share updated Figma link in WhatsApp

Notion card: "Day 3 — Stage 0 Screens" → Done + Figma link

DHANESHWARI — Day 3

Task: Student Registration API + Session Start API

Step 1: Student Registration endpoint (2 hrs)

Create backend/app/routes/student_routes.py

POST /api/students/register

Body:

```
{  
  "schoolCode": "BPT-DPS-2026",  
  "name": "Arjun Sharma",  
  "className": "11",  
  "division": "A",  
  "rollNumber": "23"  
}
```

Logic:

1. Validate schoolCode exists in schools table
→ If not: return 404 { "error": "Invalid school code", "code": "SCHOOL_NOT_FOUND" }
2. Check student not already registered (rollNumber + schoolCode combo unique)
→ If duplicate: return 409 { "error": "Student already registered", "code": "DUPLICATE_STUDENT" }
3. Get school's packageTier from schools table
4. Create student record in students table:
{ name, schoolCode, schoolId, className, division, rollNumber,
packageTier, createdAt, currentStage: 0, isActive: true }

5. Return:

```
{  
  "studentId": "uuid",  
  "name": "Arjun Sharma",  
  "schoolName": "Delhi Public School",  
  "packageTier": "starter",  
  "accessLevel": "stages_0_to_8",  
  "message": "Welcome to StartupSage, Arjun!"  
}
```

Step 2: Session Start endpoint (1.5 hrs)

POST /api/sessions/start

Body: { "studentId": "uuid", "domain": "healthcare" }

Logic:

1. Validate studentId exists
2. Check student doesn't have an active session already
 - If they do: return existing session (don't create duplicate)
3. Create simulation_sessions row:

```
{ studentId, domain, status: 'active', currentStage: 0,  
  startedAt: now(), decisions: {} }
```
4. Return: { sessionId, domain, currentStage: 0, startedAt }

Note: decisions is a JSONB column — stores all stage decisions as they come in

Step 3: Write tests for both endpoints

Add to tests/test_student.py:

→ Register with valid school code → 201 success

- Register with invalid school code → 404
- Register same student twice → 409
- Start session → 201 with sessionId
- Start second session for same student → returns existing session

SUBMISSION FOR DAY 3:

Code pushed to dhanesh/backend branch

Postman screenshot: /api/students/register success response

Postman screenshot: /api/sessions/start success response

pytest screenshot: all tests passing

WhatsApp update by 7 PM

Notion: "Day 3 — Student Registration + Session" → Done + commit link

SRUSHTI — Day 3

Task: Founder Archetype Classifier

The scenario engine is done. Now build the archetype classifier.

This runs at the end (Stage 8) to classify the student's founder type based on their pattern of decisions across all stages.

File: ai/scoring/archetype_classifier.py

Step 1: Define the 5 archetypes with scoring dimensions (45 min)

```
ARCHETYPES = {
```

```
    'visionary': {
```

```
        'name': 'The Visionary 🦊',
```

```
        'description': 'You see what others don\'t. You chose an ambitious problem, '
```



```
'thought long-term, and made bold decisions at the crossroads. '
'Your challenge: execution. Ideas need builders.',
'dimensions': ['creativity', 'innovation', 'decision_making']
},
'hustler': {
  'name': 'The Hustler 🏆',
  'description': 'You move fast and sell hard. You allocated budget to marketing, '
    'chose B2C, and kept pushing even when things got difficult. '
  'Your challenge: building systems that scale without you.',
  'dimensions': ['execution', 'resilience', 'confidence']
},
'strategist': {
  'name': 'The Strategist 🧠',
  'description': 'You think before you act. You validated carefully, '
    'chose your team wisely, and planned your business model thoroughly. '
  'Your challenge: speed. Good enough now beats perfect later.',
  'dimensions': ['decision_making', 'problem_solving', 'creativity']
},
'community_builder': {
  'name': 'The Community Builder 🤝',
  'description': 'You bring people together. You invested in your team, '
    'negotiated fairly on equity, and thought about the human side '
    'of building. Your challenge: not everyone thinks like you.',
  'dimensions': ['teamwork', 'communication', 'leadership']
},
'operator': {
  'name': 'The Operator 🖨️',
  'description': 'You build systems that scale. Your resource allocation was '
```

```

        'disciplined, your monthly decisions were consistent, '
        'and you managed through the grind. '
        'Your challenge: seeing the big picture beyond the numbers.',
        'dimensions': ['execution', 'consistency', 'decision_making']
    }
}

```

Step 2: Write the classifier function (1.5 hrs)

```
def classify_archetype(decisions: dict, emotional_checkins: list) -> dict:
```

```
    """
```

```
    Classifies student into one of 5 founder archetypes.
```

Args:

decisions: full decisions dict from all stages

emotional_checkins: list of 3 emotional responses (one per check-in)

Returns:

```

    {
        archetype_id: str,
        archetype_name: str,
        archetype_description: str,
        strengths: list[str], # top 3 scoring dimensions
        growth_areas: list[str] # bottom 2 scoring dimensions
    }
    """

```

```
# Score each archetype 0-100 based on decision patterns
```

```
scores = {
```

```
'visionary': 0,  
'hustler': 0,  
'strategist': 0,  
'community_builder': 0,  
'operator': 0  
}
```

```
domain = decisions.get('domain', '')  
co_founder = decisions.get('co_founder_choice', '')  
marketing_pct = decisions.get('budget_allocation', {}).get('marketing', 0)  
tech_pct = decisions.get('budget_allocation', {}).get('tech', 0)  
model = decisions.get('business_model', '')  
mvp_type = decisions.get('mvp_type', '')  
stage7_path = decisions.get('stage7_path', '')  
equity = decisions.get('equity_split', 50)
```

```
# Visionary signals
```

```
if stage7_path == 'raise_funding': scores['visionary'] += 30  
if domain in ('ai', 'environment'): scores['visionary'] += 20  
if mvp_type == 'product': scores['visionary'] += 15
```

```
# Hustler signals
```

```
if marketing_pct > 35: scores['hustler'] += 35  
if model in ('freemium', 'one_time'): scores['hustler'] += 20  
if stage7_path == 'bootstrap': scores['hustler'] += 15  
if 'Focused / Determined' in emotional_checkins: scores['hustler'] += 15
```

```
# Strategist signals
```

```

if model == 'subscription': scores['strategist'] += 25

if co_founder == 'domain_expert': scores['strategist'] += 20

if stage7_path == 'sell': scores['strategist'] += 20

if equity >= 50 and equity <= 60: scores['strategist'] += 15 # fair split


# Community Builder signals

if co_founder in ('technical', 'sales'): scores['community_builder'] += 20

if equity < 60: scores['community_builder'] += 20 # generous split

if 'Excited' in emotional_checkins: scores['community_builder'] += 20


# Operator signals

if tech_pct > 35: scores['operator'] += 25

if stage7_path == 'bootstrap': scores['operator'] += 20

if mvp_type == 'landing_page': scores['operator'] += 15


# Pick winner

archetype_id = max(scores, key=scores.get)

archetype = ARCHETYPES[archetype_id]


return {
    'archetype_id': archetype_id,
    'archetype_name': archetype['name'],
    'archetype_description': archetype['description'],
    'archetype_scores': scores,    # for debugging
    'strengths': archetype['dimensions'][:2],
    'growth_areas': [d for d in ['execution', 'communication', 'resilience']
                     if d not in archetype['dimensions']][:2]
}

```

Step 3: Write 5 tests — one per archetype (45 min)

Each test creates a decision set that should clearly produce one archetype.

Verify: correct archetype_id returned, strengths and growth_areas not empty.

Run: `pytest ai/tests/test_archetype.py -v`

All 5 must pass.

SUBMISSION FOR DAY 3:

Code pushed to srushti/ai branch

Screenshot: pytest passing (5 tests, all green)

WhatsApp update by 7 PM

Notion: "Day 3 — Archetype Classifier" → Done + commit link

DAY 4 — THURSDAY

Theme: Everyone goes deeper. Integration starts becoming visible.

YOU — Day 4

Morning: Review Day 3 submissions

Test Dhaneshwari's endpoints in your test harness HTML file

Review Ganesh's Stage 0 Figma — give specific feedback:

Does it feel like an app students will want to open?

Are tap targets big enough for thumbs? (min 44px height)

Does the emotional hook screen (Frame 5) feel meaningful or cheesy?

Review Srushti's archetype tests — pick one archetype manually,

trace through the scoring logic, does it feel right?

Afternoon: Build Save Stage Decisions endpoint (you write this one)

This is the most critical API in the whole simulation.

Every time a student makes a decision, this gets called.

POST /api/sessions/:sessionId/save-stage

Body:

```
{
  "stage": 1,
  "decisions": {
    "chosen_idea_index": 2,
    "customer_reaction": "interested",
    "idea_description": "An app that connects rural students to city tutors"
  },
  "emotionalCheckIn": "Excited" ← only present for stages 3, 5, 6
}
```

Logic:

1. Fetch session by sessionId — verify it belongs to the requesting student
2. Verify stage is the correct next stage (prevent skipping)
3. Merge new decisions into session.decisions JSONB:
decisions = { ...existing, stage_{N}: { ...new_decisions } }
4. Update session.currentStage = stage + 1
5. If stage == 8: update session.status = 'completed', completedAt = now()
6. Return: { saved: true, nextStage: stage + 1, sessionId }

This endpoint is called after every stage. Get it right.

Write 4 tests: save stage 0, save stage 5 with emotional check-in,

try to save wrong stage order (should fail), save stage 8 (marks complete).

GANESH — Day 4

Task: Design Stage 1 (The Idea) — All States + Reusable Decision Component

Stage 1 is where students generate and validate their startup idea.

The virtual customer reaction is the key element — design it memorably.

Frame 1: Stage 1 — Idea Generation (3 idea cards)

Top: Progress dots (Stage 0 complete = orange filled, Stage 1 current = pulsing)

Stage label: "Stage 1 — The Idea" (small, Text Muted)

Headline: "Ideas are easy. Good ideas are rare." (H2)

Sub: "Here are 3 paths you could take. Which one feels right?" (Body, Text Secondary)

3 idea cards stacked vertically (use Card component):

Each card top: idea type chip (App / Marketplace / Service / Platform)

Idea title (Body/Large, SemiBold)

2-line description of the idea concept

Bottom section (separated by divider):

"Virtual Customer Says:" label (Body/Small, Text Muted)

Customer reaction badge — design 3 states:

 Skeptical: Red background chip, "I wouldn't pay for this"

 Neutral: Yellow background chip, "Maybe if it were different"

 Interested: Green background chip, "Tell me more"

1-sentence customer quote in italics (Text Secondary)

Frame 2: Idea Selected State

One card has: orange border 2px, light orange bg tint

Other 2 cards: slightly reduced opacity (85%)

[Pursue this idea →] button: active (BPT Orange, full width)

Frame 3: Decision Component (Reusable)

This component is used in EVERY stage for every decision.

Design it as a standalone component in your Figma component library.

Structure:

Option text (Body/Large)

Sub-description (Body/Small, Text Secondary, optional)

Right side: radio circle (unselected: grey border, selected: orange fill + white dot)

Selected state: light orange left border accent (4px), light orange bg tint

States: Unselected / Selected / Disabled

Used in: Stage 1 (idea choice), Stage 2 (pricing model),

Stage 3 (negotiation), Stage 7 (path choice)

Frame 4: Emotional Check-in Component (Reusable)

This appears after Stages 3, 5, and 6.

Design as standalone component:

Card with soft background (not white — slightly warm grey: #F5F4F3)

Top: emoji that fits the question (🤔 for Stage 3, 🧐 for Stage 5)

Prompt text (Body/Large, centered, max 2 lines)

5 emotion pill options in a Wrap layout:

Default: white bg, border, Text Primary

Selected: BPT Orange bg, white text

[Continue] button below (full width, BPT Orange, disabled until selection)

SUBMISSION FOR DAY 4:

Stage 1 frames (all 4 above) in Figma

Decision Component and Emotional Check-in Component

added to Figma component library

Updated Figma link in WhatsApp

Notion: "Day 4 — Stage 1 + Reusable Components" → Done + Figma link

DHANESHWARI — Day 4

Task: Save Stage Decisions + Get Progress APIs

Wait — you just saw that the CTO (Bug) is building Save Stage today.

Your job today: read his endpoint spec carefully and build the

Get Progress endpoint + the Scenario Trigger endpoint that calls Srushti's module.

Step 1: Get Session Progress endpoint (1.5 hrs)

GET /api/sessions/:sessionId/progress

Logic:

1. Fetch session by sessionId
2. Fetch all related stage_decisions for this session
3. Return full picture:

```
{  
  "sessionId": "uuid",  
  "domain": "healthcare",  
  "currentStage": 3,  
  "status": "active",  
  "startedAt": "2026-07-14T10:00:00Z",
```

```

"decisions": {
  "stage_0": { "domain": "healthcare", "problem_index": 2 },
  "stage_1": { "chosen_idea": 0, "customer_reaction": "interested" },
  "stage_2": { "business_model": "subscription", "reality_check_response": 1 }
},
"emotionalCheckIns": [],
"percentComplete": 33
}

```

$\text{percentComplete} = (\text{currentStage} / 9) * 100$, rounded to nearest int

Step 2: Trigger Scenario endpoint (calls Srushti's module) (2 hrs)

POST /api/scenarios/assign

Body: { "sessionId": "uuid" }

Logic:

1. Fetch session + all decisions

2. Build the decisions dict that Srushti's function expects:

```

{ domain, co_founder_choice, equity_split, mvp_type,
  budget_allocation, business_model }

```

(extract from the session's stored decisions JSONB)

3. Import Srushti's function:

```

import sys

sys.path.append('../ai') # path to ai folder

from scenario_engine.scenario_engine import assign_wall_scenario

```

4. Call it: `result = assign_wall_scenario(decisions_dict)`

5. Store result in session:

Update sessions table: `wall_scenario = result`

6. Return: { scenarioId, scenarioTitle, scenarioDescription }

Note to Dhaneshwari: coordinate with Srushti today.

Make sure the function import path works on your machine.

If it doesn't — message in WhatsApp group, solve it together.

Step 3: Test the trigger endpoint

Create a test session with stage 0–4 decisions pre-loaded

Call POST /api/scenarios/assign

Verify: correct scenario fires (match Srushti's test cases)

SUBMISSION FOR DAY 4:

Code pushed to dhanesh/backend branch

Postman screenshots: GET /progress and POST /scenarios/assign both working

WhatsApp update by 7 PM

Notion: "Day 4 — Progress + Scenario Trigger" → Done + commit link

SRUSHTI — Day 4

Task: Entrepreneurial Skills Score — First 5 Dimensions

The Skills Score computation is the most complex module you'll write.

10 dimensions total. Build the first 5 today, last 5 on Day 5.

File: ai/scoring/skills_score.py

Step 1: Define the scoring structure (30 min)

```
def compute_skills_score(decisions: dict, emotional_checkins: list,  
                        offline_challenge_complete: list) -> dict:
```

'''

Computes a 0-100 score across 10 entrepreneurial skill dimensions.

Args:

decisions: full decisions dict from all 9 stages

emotional_checkins: list of 3 emotion strings (one per check-in)

offline_challenge_complete: list of 9 bools (one per stage)

Returns:

dict with score per dimension + overall_founder_score

'''

Step 2: Write scoring for first 5 dimensions (3 hrs)

DIMENSION 1: CONFIDENCE (0-100)

How boldly did they commit to their idea and defend it?

def score_confidence(decisions):

score = 50 # baseline

Strong problem conviction in Stage 0 (picked quickly, didn't second-guess)

if decisions.get('stage_0', {}).get('time_to_decide_seconds', 60) < 30:

score += 15

Chose an ambitious Stage 7 path (raise funding or scale)

if decisions.get('stage7_path') == 'raise_funding':

score += 20

Didn't quit in Stage 5

if decisions.get('stage_5', {}).get('wall_choice') != 'quit':

score += 15

return min(score, 100)

```
# DIMENSION 2: LEADERSHIP (0-100)
```

```
# How well did they manage their team?
```

```
def score_leadership(decisions):
```

```
    score = 50
```

```
    co_founder = decisions.get('co_founder_choice', '')
```

```
    equity = decisions.get('equity_split', 50)
```

```
    stage3_negotiation = decisions.get('stage_3', {}).get('negotiation_response', '')
```

```
    # Chose a co-founder (shows they can lead with others)
```

```
    if co_founder != 'solo':
```

```
        score += 15
```

```
    # Fair equity split (45-55 range shows collaborative leadership)
```

```
    if 45 <= equity <= 55:
```

```
        score += 20
```

```
    # Handled negotiation well
```

```
    if stage3_negotiation == 'collaborative':
```

```
        score += 15
```

```
    return min(score, 100)
```

```
# DIMENSION 3: COMMUNICATION (0-100)
```

```
# Did they communicate effectively with customers and team?
```

```
def score_communication(decisions):
```

```
    score = 50
```

```
    # Stage 1: customer reaction quality
```

```
    reaction = decisions.get('stage_1', {}).get('customer_reaction', '')
```

```
    if reaction == 'interested':
```

```
        score += 25
```

```

elif reaction == 'neutral':

    score += 10

# Emotional check-in 1 shows open communication
if emotional_checkins and emotional_checkins[0] in ('Excited', 'Confident'):

    score += 15

# Stage 3 negotiation response
if decisions.get('stage_3', {}).get('negotiation_response') == 'direct':

    score += 10

return min(score, 100)


# DIMENSION 4: PROBLEM SOLVING (0-100)
# How did they respond to obstacles?
def score_problem_solving(decisions):

    score = 50

    wall_choice = decisions.get('stage_5', {}).get('wall_choice', "")

    # Pivoted or fought = active problem solving
    if wall_choice in ('fight', 'pivot'):

        score += 30

    elif wall_choice == 'pause':

        score += 15

    # Stage 6: quality of monthly decisions
    stage6_decisions = decisions.get('stage_6', {}).get('monthly_choices', [])
    good_choices = sum(1 for c in stage6_decisions if c.get('quality') == 'good')
    score += min(good_choices * 5, 20)

    return min(score, 100)


# DIMENSION 5: CREATIVITY (0-100)
# Did they take creative approaches?

```

```
def score_creativity(decisions):  
    score = 50  
    mvp_type = decisions.get('mvp_type', '')  
    # Paper prototype or Canva = creative over technical  
    if mvp_type in ('paper', 'canva'):  
        score += 20  
    # Chose environment or AI domain (less obvious choices)  
    if decisions.get('domain') in ('environment', 'ai'):  
        score += 15  
    # Creative business model  
    if decisions.get('business_model') == 'freemium':  
        score += 15  
    return min(score, 100)
```

Step 3: Write 3 tests covering all 5 dimensions

Test with a "confident creative" student profile

Test with a "cautious operator" profile

Test with a "solo struggling" profile

Verify: scores feel right for the profiles (not just that they return numbers)

Run: `pytest ai/tests/test_skills_score.py -v`

SUBMISSION FOR DAY 4:

Code pushed to srushti/ai branch (skills_score.py with first 5 dimensions)

Screenshot: pytest passing

WhatsApp update by 7 PM

Notion: "Day 4 — Skills Score (5 Dimensions)" → Done + commit link

DAY 5 — FRIDAY

Theme: Wrap Week 1. Demo day. Set Week 2 tasks.

YOU — Day 5

Morning:

Review all Day 4 submissions

Final code review of all branches:

ghanesh/backend: pull the branch, run the APIs locally, spot any issues

srushti/ai: run all tests, check the scoring logic makes sense

ganesh/design: review all Figma frames, check spacing consistency

Create PR from dev → your review (not merging yet, just reviewing state)

Afternoon: Week 1 Demo Call (1 hr, all 4 together)

Schedule for 4 PM

Format (15 min each intern):

Ganesh: shares screen, walks through Figma — Stage 0 and Stage 1 all states

Dhaneshwari: shares Postman, demos all 4 endpoints live

Srushti: shares terminal, runs all tests live (scenario engine + archetype + 5 skills)

You: give feedback on each demo

Don't skip this call — it's the most important accountability moment of the week

After the call: Write Week 2 tasks in Notion

(Assign tasks for next Monday before you sign off Friday)

This ensures Monday morning everyone knows what they're doing without waiting for you to brief them.

Post in WhatsApp group:

"Week 1 done. Good start. Here's what I noticed: [specific observations]

Week 2 tasks are in Notion. Monday brief call at 10 AM before you start."

GANESH — Day 5

Task: Design Stage 2 (The Plan) + Week 1 cleanup

Frame 1: Business Model Canvas — Empty State

Top: progress dots, stage label "Stage 2 — The Plan"

Headline: "Dreams become plans." (H2)

Sub: "Build your startup's business model." (Body, Text Secondary)

6 input cards (use Card component, vertical stack):

Each card:

Field label (Body/Small, SemiBold, Text Primary)

Hint text inside (italic, Text Muted)

On focus: orange border 1.5px

Fields + hints:

"Who is your customer?" — "Be specific. Not 'everyone'. A real person."

"What problem do you solve?" — "One sentence. The simpler the better."

"How do you reach them?" — "Instagram? Schools? Hospitals? WhatsApp?"

"How do you make money?" — "Will they pay? When? How much?"

"What does it cost to get one customer?" — "Ads? Calls? Events? Estimate."

"What do you charge per customer?" — "₹___ per month or per transaction"

Bottom: Business model selector

3 pill options (use your Decision Component):

 Subscription |  One-time purchase |  Freemium

Frame 2: Canvas Filled State

Same layout but all 6 fields have placeholder typed text

Business model: "Subscription" selected (orange pill)

[Submit my plan] button: active

Frame 3: AI Reality Check (appears after submit)

Animated card slides up from bottom (note: "slide up animation")

Card design:

Top: small robot emoji + "Reality Check" label (Text Muted)

Divider

Challenge question in bold: "Your CAC is ₹800 but you charge ₹299.

You lose ₹501 on every customer. What do you change?"

Below: 3 Decision Component options:

A) Raise my price significantly

B) Find a cheaper acquisition channel

C) Change to a different business model

[Confirm my answer] button

Frame 4: Stage 2 Complete

Brief "lesson learned" moment before Stage 3:

Icon: lightbulb (or similar)

"What you just did:" label

"A Business Model Canvas. Most founders skip this. The ones who don't, survive longer." (quote style, centered)

[Continue to The Team →] button

Week 1 cleanup tasks:

Go back through ALL Figma frames from Days 2–5

Check every frame uses your design system components (not custom one-offs)

Check typography: are you using the text styles or typing sizes manually?

Check spacing: use 8px grid throughout (padding always multiples of 8)

Export all frames as PNG (1x) for reference

SUBMISSION FOR DAY 5:

Stage 2 frames (all 4) + cleaned up all previous frames

Figma prototype: link Stage 0 → Stage 1 → Stage 2 with clickable arrows

(simple click-through prototype, not full interaction)

Share updated Figma link in WhatsApp

Notion: "Day 5 — Stage 2 + Week 1 Cleanup" → Done + Figma link

Also: attend 4 PM demo call. Be ready to walk through Figma.

DHANESHWARI — Day 5

Task: Offline Challenge Submission API + Week 1 cleanup + PR prep

Step 1: Offline Challenge Submission endpoint (2 hrs)

POST /api/challenges/submit

Body (multipart/form-data or JSON):

```
{  
  "sessionId": "uuid",  
  "stage": 0,  
  "challengeType": "find_5_problems",
```

```
"proofUrl": "https://supabase.co/storage/...", ← uploaded separately
"textNotes": "I found these 5 problems: ..." ← optional
}
```

Note: File upload to Supabase Storage is handled on the frontend (Flutter).

Flutter uploads the file, gets back a URL, then sends that URL here.

This endpoint only stores the record, not the file.

Logic:

1. Validate sessionId + stage exist
2. Check stage matches a valid challenge stage (0–8)
3. Insert into offline_challenge_submissions:

```
{ sessionId, studentId, stage, challengeType, proofUrl,
  textNotes, submittedAt, aiFeedback: null, badgeEarned: null }
```
4. Return: { submissionId, message: "Challenge submitted! AI feedback coming soon." }

Note: AI feedback is async — Srushti's module will be called by a background job (not in this endpoint). For now, return without feedback.

We add the async AI feedback call in Week 2.

Step 2: Week 1 code cleanup (1 hr)

Go through all your code from Days 1–4:

→ Remove any print() debug statements (replace with proper logging)

Use Python's logging module:

```
import logging

logger = logging.getLogger(__name__)

logger.info(f"Student registered: {student_id}")
```

```
logger.error(f"Registration failed: {str(e)}")
```

- Add docstrings to every function (3 lines: what it does, args, returns)
- Make sure .env keys are never hardcoded (use os.getenv() everywhere)
- Check: are all error responses consistent format? { error, code, status }

Step 3: Raise PR (30 min)

Create a Pull Request: dhanesh/backend → dev

PR title: "Week 1 Backend — School + Student + Session + Challenge APIs"

PR description:

What I built

- POST /api/admin/schools — create school with auto code generation
- POST /api/students/register — student registration with school code validation
- POST /api/sessions/start — simulation session creation
- GET /api/sessions/:id/progress — full session state retrieval
- POST /api/scenarios/assign — wall scenario trigger (calls Srushti's module)
- POST /api/challenges/submit — offline challenge submission

How to test

Import the Postman collection: [attach file or link]

Run: pytest tests/ -v (all X tests pass)

Known issues / not done yet

AI feedback call for challenge submission (Week 2)

SUBMISSION FOR DAY 5:

Code pushed to dhanesh/backend

PR raised → share PR link in WhatsApp

Postman collection exported as JSON → uploaded to Notion

WhatsApp update by 7 PM

Attend 4 PM demo call

Notion: "Day 5 — Challenge API + PR" → Done + PR link

SRUSHTI — Day 5

Task: Skills Score — Last 5 Dimensions + Full Module Integration

Step 1: Write last 5 dimensions in skills_score.py (2 hrs)

DIMENSION 6: INNOVATION (0-100)

```
def score_innovation(decisions):  
    score = 50  
  
    if decisions.get('business_model') == 'freemium': score += 15  
    if decisions.get('domain') in ('ai', 'environment'): score += 20  
    if decisions.get('stage7_path') == 'raise_funding': score += 15  
    return min(score, 100)
```

DIMENSION 7: TEAMWORK (0-100)

```
def score_teamwork(decisions):  
    score = 50  
  
    co_founder = decisions.get('co_founder_choice', "")  
    if co_founder != 'solo': score += 20  
  
    equity = decisions.get('equity_split', 50)  
    if 45 <= equity <= 60: score += 15 # generous split  
  
    stage6_team = decisions.get('stage_6', {}).get('team_decision', "")  
    if stage6_team == 'gave_raise': score += 15  
    return min(score, 100)
```

DIMENSION 8: EXECUTION (0-100)

```
def score_execution(decisions):  
    score = 50  
  
    budget = decisions.get('budget_allocation', {})  
  
    tech_pct = budget.get('tech', 0)  
    marketing_pct = budget.get('marketing', 0)  
    ops_pct = budget.get('ops', 0)  
  
    # Balanced allocation (no single area above 50%)  
    if all(v <= 50 for v in [tech_pct, marketing_pct, ops_pct]):  
        score += 25  
  
    mvp_type = decisions.get('mvp_type', '')  
    if mvp_type in ('landing_page', 'product'): score += 15  
  
    offline_done = sum(1 for x in offline_challenge_complete if x)  
    score += min(offline_done * 3, 15)  
  
    return min(score, 100)
```

DIMENSION 9: RESILIENCE (0-100)

```
def score_resilience(decisions, emotional_checkins):  
    score = 50  
  
    wall_choice = decisions.get('stage_5', {}).get('wall_choice', '')  
    if wall_choice in ('fight', 'pivot'): score += 30  
    elif wall_choice == 'pause': score += 10  
  
    if len(emotional_checkins) > 1:  
        checkin2 = emotional_checkins[1] # Stage 5 check-in  
        if checkin2 in ('Focused', 'Surprisingly calm'): score += 20  
  
    return min(score, 100)
```

DIMENSION 10: DECISION MAKING (0-100)

```

def score_decision_making(decisions):
    score = 50
    stage7_path = decisions.get('stage7_path', '')
    if stage7_path in ('raise_funding', 'bootstrap'): score += 25 # decisive
    elif stage7_path == 'sell': score += 15
    # Stage 6 consistency: did they stick to a strategy across months?
    stage6 = decisions.get('stage_6', {})
    if stage6.get('strategy_consistent', False): score += 25
    return min(score, 100)

```

Step 2: Integrate all 10 dimensions + compute overall score (1 hr)

Complete the compute_skills_score() function:

```

confidence = score_confidence(decisions)
leadership = score_leadership(decisions)
communication = score_communication(decisions)
problem_solving = score_problem_solving(decisions)
creativity = score_creativity(decisions)
innovation = score_innovation(decisions)
teamwork = score_teamwork(decisions)
execution = score_execution(decisions, offline_challenge_complete)
resilience = score_resilience(decisions, emotional_checkins)
decision_making = score_decision_making(decisions)

# Weighted overall score (resilience, execution, decision_making weighted 1.3x)
weights = {
    'confidence': 1.0, 'leadership': 1.0, 'communication': 1.0,
    'problem_solving': 1.0, 'creativity': 1.0, 'innovation': 1.0,
    'teamwork': 1.0, 'execution': 1.3, 'resilience': 1.3, 'decision_making': 1.3
}

```



```

}

scores = { ... } # dict of all 10

weighted_sum = sum(scores[k] * weights[k] for k in scores)

total_weight = sum(weights.values())

overall = round(weighted_sum / total_weight)


return { ...all 10 dimensions..., 'overall_founder_score': overall }

```

Step 3: Write 3 integration tests + raise PR (1 hr)

Test 1: "Well-rounded founder" — all good decisions → overall score 75-90

Test 2: "Solo struggling founder" — hard choices → score 40-60

Test 3: "Operator type" — disciplined budget, consistent → high execution score

All tests pass → raise PR:

PR: srushti/ai → dev

Title: "Week 1 AI Modules — Scenario Engine + Archetype + Skills Score"

Description: what each module does, inputs/outputs, test count

SUBMISSION FOR DAY 5:

Complete skills_score.py pushed to srushti/ai

Screenshot: full pytest run (all tests from all 3 modules passing)

PR raised → share link in WhatsApp

Attend 4 PM demo call

Notion: "Day 5 — Skills Score Complete + PR" → Done + PR link

WEEK 1 END — WHAT YOU SHOULD HAVE

YOU:

- ✓ GitHub repo created + branch structure
- ✓ Supabase project + all 10 tables created
- ✓ .env.example + CONTRIBUTING.md in repo
- ✓ Internal test harness HTML file working
- ✓ POST /api/sessions/:id/save-stage built and tested
- ✓ Demo call held + Week 2 tasks in Notion

GANESH:

- ✓ Design system: colours, typography, 8 reusable components
- ✓ Stage 0: 5 frames (Splash, Domain select, Problem discovery, Emotional hook)
- ✓ Stage 1: 4 frames (3 ideas, selected, customer reactions)
- ✓ Stage 2: 4 frames (Canvas, filled, reality check, lesson)
- ✓ Clickable prototype: Stage 0 → 1 → 2 linked
- ✓ All frames consistent with design system

DHANESHWARI:

- ✓ Flask app running locally
- ✓ Supabase connection working
- ✓ 6 endpoints built + tested:
POST /admin/schools | POST /students/register | POST /sessions/start
GET /sessions/:id/progress | POST /scenarios/assign | POST /challenges/submit
- ✓ All tests passing
- ✓ PR raised to dev branch

SRUSHTI:

- ✓ scenario_engine.py — 8 scenarios, 10 unit tests passing

- ✅ archetype_classifier.py — 5 archetypes, 5 tests passing
 - ✅ skills_score.py — 10 dimensions, 3 integration tests passing
 - ✅ PR raised to dev branch
-

WEEK 2 PREVIEW (Tell Everyone Friday)

Ganesh: Stages 3, 4, 5, 6 in Figma (Team, Build, Wall, Grind)

Dhanesh: School Dashboard API + Sunday session management + tests

Srushti: Claude API integration (AI customer conversation + challenge feedback)

You: Merge Week 1 PRs + Flutter project setup for Ganesh +

BPT Phase 2 (AI matching, afternoons)

StartupSage Phase 1 — Week 1 Task List You + Ganesh + Dhaneshwari + Srushti | Day by day | July 2026